Tabla de contenidos

1. Funciones *built-in* en Python
    a. `len`
    b. `__getitem__`
    c. `reversed`
    d. Otras funciones *built-in*
2. Librerías *built-in*
    a. `itertools`
    b. `collections`
3. Casos aplicados
    a. Consultas anidadas
    b. Unir generadores
    c. Eficiencia (Generadores vs Estructuras que usan memoria)

Funciones *built-in* en Python

Existen muchas funciones que vienen implementadas en Python, principalmente con el propósito de simplificar y abstraer cálculos que pueden aplicar a objetos de clases distintas (*duck typing*). Pueden revisar todas ellas en la documentación de funciones de Python. Veamos algunos ejemplos:

`len`

Retorna el número de elementos que posee un contenedor, como por ejemplo una lista, un diccionario, un *set*, etc.

```
print(len([3, 4, 1, 5, 5, 2]))
print(len({"nombre": "Juan", "apellido": "Martínez"}))
```

```
6
2
```

La función `len()` aplicada a un objeto en particular (`objeto`) hace un llamado a `objeto.__len__()`. La función `__len__` viene implementada en varias clases de estructuras de datos *built-in*.

Podemos ver que llamando a `objeto.__len__()` directamente obtenemos el mismo resultado que a través de `len(objeto)`

```
print([3, 4, 1, 5, 5, 2].__len__())
print({"nombre": "Juan", "apellido": "Martínez"}.__len__())
```

```
6
2
```

También se puede hacer *overriding* del método `__len__`. Supongamos que queremos implementar un tipo especial de lista cuyo método `__len__` retorna el largo de la lista sin considerar los elementos que se repiten:

```
class MiLista(list):
    """
    Tipo especial de lista, donde len(lista)
    retorna el largo sin considerar repetidos
    """
    def __len__(self) -> int:
        # Creamos un set con los datos que tenemos
        datos_sin_repetir = set(self)

        # Retornamos el largo de este set aprovechando que elimina los
repetidos
        return len(datos_sin_repetir)

mi_lista = MiLista([1, 2, 3, 4, 5, 6, 6, 7, 7, 7, 7, 2, 2, 3, 3, 1,
1])
print(len(mi_lista))
```

```
7
```

`__getitem__`

Al definir esta función dentro de una clase, podemos acceder a los elementos mediante algún tipo de índice o llave usando la notación `objeto[valor]`. Esto da paso a dos comportamientos que podemos emular: una secuencia (donde hay un valor después de otro de manera ordenada, como una lista) o un *mapping* (donde hay llaves que permiten acceder a valores, como un diccionario).

En el primer caso, donde queremos que nuestra clase se comporte como una secuencia, el método `__getitem__` debería recibir enteros (`int`) y lanzar la excepción `IndexError` si es que el índice no es válido.

```
class MiString:
    def __init__(self, palabra: str) -> None:
        self.palabra = palabra

    def __getitem__(self, i: int) -> str:
        print(f"Pidiendo el elemento {i}:")
        return self.palabra[i]

mi_string = MiString("Mundo")
mi_string[0]
mi_string[15]
```

```
Pidiendo el elemento 0:
Pidiendo el elemento 15:
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call
last)
Cell In[15], line 11
      9 mi_string = MiString("Mundo")
     10 mi_string[0]
---> 11 mi_string[15]

Cell In[15], line 7, in MiString.__getitem__(self, i)
      5 def __getitem__(self, i: int) -> str:
      6     print(f"Pidiendo el elemento {i}:")
----> 7     return self.palabra[i]

IndexError: string index out of range
```

Definir `__getitem__`, para que nuestra clase se comporte como una secuencia, nos permite iterar sobre la estructura mediante un `for`, es decir, el objeto será **iterable**. En este caso, el `for` irá pidiendo los elementos desde el 0 en adelante hasta que se lance una excepción. En nuestro ejemplo, esto nos permite iterar sobre la palabra completa. Al intentar acceder fuera del largo de la palabra que estamos guardando, se lanza una exepción de tipo `IndexError` que detendrá el `for`.

```
for caracter in mi_string:
    print(caracter, end='\n\n')
```

```
Pidiendo el elemento 0:
M

Pidiendo el elemento 1:
u

Pidiendo el elemento 2:
n

Pidiendo el elemento 3:
d

Pidiendo el elemento 4:
o

Pidiendo el elemento 5:
```

En el caso de que queramos que nuestra clase se comporte como una estructura de tipo *mapping*, podemos usar cualquier tipo de llave. Si una llave es del tipo equivocado, debería lanzarse la excepción `TypeError`, mientras que si es del tipo correcto pero la llave no existe debería lanzarse la excepción `KeyError`.

El siguiente ejemplo es similar al primero, pero las llaves son *strings* de una palabra y el valor retornado es la cantidad de veces que aparece dicho *string* en la palabra. Como habrás notado,

ahora nuestra función `__getitem__` debería recibir un *string*, lanzando la excepción `TypeError` si no es así.

```
class ContadorLetras:
    def __init__(self, palabra: str) -> None:
        self.palabra = palabra

    def __getitem__(self, key: str) -> int:
        print(f"Pidiendo el elemento {key}:")
        # Notar que puedes decir levantar una excepción si la llave no
está
        # if key not in self.palabra:
        #     raise KeyError("La letra no está en la palabra")
        return self.palabra.count(key)

contador_letras = ContadorLetras("Hola-Mundo")
contador_letras["o"]
```

```
Pidiendo el elemento o:

2

contador_letras[3]

Pidiendo el elemento 3:

---------------------------------------------------------------------------
-----
TypeError                                 Traceback (most recent call
last)
Cell In[18], line 1
----> 1 contador_letras[3]

Cell In[17], line 10, in ContadorLetras.__getitem__(self, key)
      6 print(f"Pidiendo el elemento {key}:")
      7 # Notar que puedes decir levantar una excepción si la llave no
está
      8 # if key not in self.palabra:
      9 #     raise KeyError("La letra no está en la palabra")
---> 10 return self.palabra.count(key)

TypeError: must be str, not int
```

Como supondrás, usar un `for` con nuestra implementación no funcionará, ya que al iterar sobre el objeto se utiliza el índice para acceder a los elementos.

```
for i in contador_letras:
    print(i)

Pidiendo el elemento 0:
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[19], line 1
----> 1 for i in contador_letras:
      2     print(i)

Cell In[17], line 10, in ContadorLetras.__getitem__(self, key)
      6 print(f"Pidiendo el elemento {key}:")
      7 # Notar que puedes decir levantar una excepción si la llave no está
      8 # if key not in self.palabra:
      9 #     raise KeyError("La letra no está en la palabra")
---> 10 return self.palabra.count(key)

TypeError: must be str, not int
```

`reversed`

La función `reversed()` toma una **secuencia** cualquiera como input y retorna **una copia de la secuencia** en orden inverso. También podemos personalizar la función haciendo *overriding* de `__reversed__` en cada clase.

Si no personalizamos el método `__reversed__`, se usará el *built-in* que iterará usando `__getitem__` y `__len__`. En ese caso, se itera `__len__` veces sobre el objeto usando `__getitem__` hacia atrás.

Por ejemplo, podemos definir un tipo especial de lista que hace *override* de `__reversed__`. En este caso, intercambia la primera mitad con la segunda, en vez de invertir el orden de los elementos.

```python
class ListaReversaMitad(list):
    def __init__(self, *args) -> None:
        super().__init__(args)

    def __reversed__(self) -> list:
        mitad = len(self) // 2
        return self[mitad:] + self[:mitad]


lista = [1, 2, 3, 4, 5, 6]

for secuencia in lista, ListaReversaMitad(*lista):
    print(f"Clase {type(secuencia).__name__}: ", end="")

    for elemento in reversed(secuencia):
        print(elemento, end=", ")
    print()
```

```
Clase list: 6, 5, 4, 3, 2, 1,
Clase ListaReversaMitad: 4, 5, 6, 1, 2, 3,
```

Otras funciones *built-in*

Además de las funciones antes nombradas, Python cuenta con otras funciones built-in que también puedes ser aplicadas sobre iterables.

Hay funciones que tienen un enfoque más matemático:

- `sum()`
- `min()`
- `max()`

Y otras con un enfoque booleano:

- `all()`
- `any()`

Un caso donde es útil utilizar estas funciones cuando:

1. Queremos obtener la cantidad de elementos de un generador, sin tener que transformarlo en otra estructura de datos.

   Por defecto, los generadores no presentan el método `__len__()`, por lo que no es posible obtener su largo utilizando dicho método:

```
generador = (i for i in range(10))

largo = len(generador)
print(f'Largo generador: {largo}')
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[21], line 3
      1 generador = (i for i in range(10))
----> 3 largo = len(generador)
      4 print(f'Largo generador: {largo}')

TypeError: object of type 'generator' has no len()
```

Pero haciendo uso de la función built-in `sum` junto con `map`, es posible calcular el porte del generador:

```
generador = (i for i in range(10))

largo = sum(map(lambda x: 1, generador))
print(f'Largo generador: {largo}')
```

```
Largo generador: 10
```

1. Cuando queremos verificar si todos o alguno de los elementos de un iterble cumplen con alguna condición en específico.

```
iterable = {1, 3, 5, 11, 8, 9}

todos_son_impar = all(map(lambda x: x % 2 == 0, iterable))
contiene_un_par = any(map(lambda x: x % 2 == 1, iterable))

print(f'Todos los elementos del iterable son impares:
{todos_son_impar}')
print(f'El iterable contiene al menos un elemento par:
{contiene_un_par}')
```

```
Todos los elementos del iterable son impares: False
El iterable contiene al menos un elemento par: True
```

Librerías *built-in*

Dentro del contexto de Python, también tenemos librerías *built-in* que nos permitirán trabajar con mayor facilidad las consultas relacionadas a la programación funcional.

`itertools`

La librería *built-in* `itertools`, la cual cuenta con funciones especializadas en trabajar con iteradores. Dentro de este módulo podemos encontrar:

- Funciones que entregan iteradores infinitos:
  - `count()`
  - `cycle()`
  - `repeat()`
- Funciones orientadas en la combinatoria:
  - `product()`
  - `permutations()`
  - `combinations()`
- y muchos más...

A continuación veremos algunos ejemplos, pero te invitamos a investigar más al respecto y evaluar casos donde estas funciones puedas ser de utilidad.

1. La función `count()` entrega un generador infinito que nos permite contar desde `0` aumentando los valores en `1`. Esta función nos permite cambiar el valor desde el que queremos contar (`start`) o el valor que será utilizado para aumentar el contador (`step`).

```
from itertools import count

contador = count(10, 0.5)
```

```
for _ in range(6):
    print(next(contador))
```

```
10
10.5
11.0
11.5
12.0
12.5
```

1. La función `combinations()` recibe un iterable y número entero `r`, entrega un generador con la combinatoria de los distintos elementos del iterable, donde cada combinación entregada es de porte `r`.

```
from itertools import combinations

combinaciones_grupos_2 = combinations('ABCD', 2)
combinaciones_grupos_4 = combinations('ABCD', 4)

print(list(combinaciones_grupos_2))
print(list(combinaciones_grupos_4))
```

```
[('A', 'B'), ('A', 'C'), ('A', 'D'), ('B', 'C'), ('B', 'D'), ('C',
'D')]
[('A', 'B', 'C', 'D')]
```

`collections`

Por otro lado, la librería *built-in* `collections` nos entrega alternativas a *dicts*, *lists*, *sets* y *tuples*.

Entre los métodos y objetos que ya hemos visto podemos encontrar:

- `namedtuple()`
- `deque`
- `defaultdict`

Pero también cuenta con otros objetos como:

- `Counter`

Veamos un ejemplos:

1. El objeto `Counter` es una subclase de diccionario que permite contar los elementos de un iterable. Cada elementos será almacenado como llave del diccionario y sus recuentos, como valores del mismo.

   Es importante tener en consideración que para que `Counter` funcione correctamente, los elementos del iterable deben ser hasheables.

```
from collections import Counter
from random import randint

iterable = [randint(1, 3) for _ in range(20)]
contador = Counter(iterable)
contador
```

```
Counter({2: 8, 1: 7, 3: 5})
```

Además, el objeto `Counter` no solo nos permite obtener la cantidad de veces que se repite cada elemento del iterable, también lo podemos utilizar para encontrar los elementos más repetidos dentro del mismo:

```
texto = 'supercalifragilisticoespialidoso'
contador = Counter(texto)  # Recuerda que los strings también son iterables
contador.most_common(3)
```

```
[('i', 6), ('s', 4), ('a', 3)]
```

Casos aplicados

Finalmente, para ir cerrando este contenido, veamos algunos casos aplicados donde se haga uso de los contenidos vistos hasta el momento.

Consultas anidadas

Una de las grandes características que presenta la programación funcional, es que podemos anidar las consultas que se realizan, siempre y cuando estas retornen un generador o algún tipo de estructura iterable.

Por ejemplo, asumiendo que tenemos un generador con información de distintas lenguas extintas o en peligro de extinción, donde cada grupo de datos contiene:

- ID
- Nombre de la lengua
- Países donde se habla (separador por ;)
- El código de los países donde se habla (separador por ;)
- El grado de peligro
- La cantidad de hablantes

```
from collections import namedtuple
from typing import Generator


LenguaExtinta = namedtuple('LenguaExtinta', 'id, name, country, country_code, endangerment, speakers')

def generador_lenguas_extintas() -> Generator:
```

```
    yield LenguaExtinta(349, 'Juang', 'India', 'IND', 'Definitely
endangered', 230000)
    yield LenguaExtinta(350, 'Tiwa', 'India', 'IND', 'Definitely
endangered', 230000)
    yield LenguaExtinta(351, 'Embera (Panama)','Panama', 'PAN',
'Vulnerable', 224850)
    yield LenguaExtinta(352, 'Ediamat', 'Guinea-Bissau', 'GNB',
'Vulnerable', 220250)
    yield LenguaExtinta(353, 'Mara', 'India', 'IND', 'Definitely
endangered', 220000)
    yield LenguaExtinta(354, 'Maring', 'India', 'IND', 'Vulnerable',
220000)
```

A partir del generador podríamos anidar consultas para encontrar la cantidad total de hablantes de la India, que saben lenguas que se encuentran en un estado vulnerable.

```
lenguas_india = filter(lambda lengua: lengua.country == 'India',
generador_lenguas_extintas())
lenguas_vulnerables = filter(lambda lengua: lengua.endangerment ==
'Vulnerable', lenguas_india)
hablantes_totales = sum(map(lambda lengua: lengua.speakers,
lenguas_vulnerables))

print('Los hablantes totales son:', hablantes_totales)
```

```
Los hablantes totales son: 220000
```

Otra forma de haber logrado lo anterior, es haber unido consultas: ya sea por medio de las funciones lambda o de los inputs de cada una:

```
hablantes_totales = sum(
    map(lambda lengua: lengua.speakers,
        filter(lambda lengua: (lengua.country == 'India') and
(lengua.endangerment == 'Vulnerable'),
               generador_lenguas_extintas())))

print('Los hablantes totales son:', hablantes_totales)
```

```
Los hablantes totales son: 220000
```

Unir generadores

En múltiples ocasiones deberemos unir dos o generadores en base a distintas condiciones.

En este caso, trabajaremos con dos generadores de números y busquemos todos los pares que al mezclarlos sumen un múltiplo de 5.

```
from itertools import product
from random import randint
```

```
generador_1 = (randint(0, 20) for _ in range(10))
generador_2 = (randint(20, 50) for _ in range(10))

combinaciones = product(generador_1, generador_2)

# Es importante notar que 'product' nos entrega un generador que
contiene tuplas
# con los pares de elementos de ambos generadores, por lo que al
momento de querer
# aplicar una función sobre estos elementos, deberemos hacer
indexación sobre ellos
# o utilizar funciones que operen sobre iteradores.
pares_multiplos_de_5 = filter(lambda x: (x[0] + x[1]) % 5 == 0,
combinaciones)

print(list(pares_multiplos_de_5))
```

```
[(10, 50), (10, 25), (6, 29), (6, 24), (6, 39), (6, 29), (7, 48), (7,
38), (1, 29), (1, 24), (1, 39), (1, 29), (10, 50), (10, 25), (6, 29),
(6, 24), (6, 39), (6, 29), (15, 50), (15, 25), (0, 50), (0, 25), (15,
50), (15, 25)]
```

Agrupar información a través de diccionarios

A veces será necesario agrupar la información de un generador por medio de diccionarios. A continuación, evaluaremos diversas formas de lograr esto mismo.

Aprovechemos el listado de lenguas extintas y en peligro de extinción que usamos antes, y agrupemos la cantidad de lenguas que hay por país.

```
from collections import namedtuple
from typing import Generator


LenguaExtinta = namedtuple('LenguaExtinta', 'id, name, country,
country_code, endangerment, speakers')

def generador_lenguas_extintas() -> Generator:
    yield LenguaExtinta(349, 'Juang', 'India', 'IND', 'Definitely
endangered', 230000)
    yield LenguaExtinta(350, 'Tiwa', 'India', 'IND', 'Definitely
endangered', 230000)
    yield LenguaExtinta(351, 'Embera (Panama)','Panama', 'PAN',
'Vulnerable', 224850)
    yield LenguaExtinta(352, 'Ediamat', 'Guinea-Bissau', 'GNB',
'Vulnerable', 220250)
    yield LenguaExtinta(353, 'Mara', 'India', 'IND', 'Definitely
endangered', 220000)
```

```
    yield LenguaExtinta(354, 'Maring', 'India', 'IND', 'Vulnerable',
220000)
```

1. Utilizando estructuras por comprensión:

   Nota: Dado que en este ejercicio recorreremos el generador en más de una ocasión, lo transformaremos en una lista; pero esto no siempre puede ser una buena alternativa.

```
países = map(lambda lengua: lengua.country,
generador_lenguas_extintas())
lista_países = list(países)

contador = {país: lista_países.count(país) for país in
set(lista_países)}
print(contador)

{'Guinea-Bissau': 1, 'Panama': 1, 'India': 4}
```

1. Utilizando el objeto `Counter` de `collections`:

```
from collections import Counter

países = map(lambda lengua: lengua.country,
generador_lenguas_extintas())
contador = Counter(países)
print(contador)

Counter({'India': 4, 'Panama': 1, 'Guinea-Bissau': 1})
```

1. Utilizando `reduce`:

```
from functools import reduce


def función_auxiliar(diccionario, lengua) -> dict:
    país = lengua.country

    if país not in diccionario:
        diccionario[país] = 0

    diccionario[lengua.country] += 1
    return diccionario

contador = reduce(función_auxiliar, generador_lenguas_extintas(), {})
print(contador)

{'India': 4, 'Panama': 1, 'Guinea-Bissau': 1}
```

Eficiencia (Generadores vs Estructuras que usan memoria)

Finalmente, realicemos algunos ejemplos aplicados donde comparemos la realizar consultas utilizando unicamente generadores vs estructuras que utilizan memoria (*lists*, *dicts*, *tuples*, *sets*, ...).

Para esto, utilizaremos un *dataset* de transacciones que tarjetas de crédito, el cual cuenta con más de 1,2 millones de datos y pesa más de 300MB.

Consideraciones

> Debido a que los códigos de esta sección están limitados para ser ejecutados en un tiempo límite, en este *notebook* encontrarás los códigos implementados como texto y los resultados de la ejecución de los mismos. Estos fueron ejecutados usando `python -B -m unittest -v`; el `-B` para que no se use el caché y así tener valores mas comparables.
>
> Si quieres ejecutar los códigos en tu computador, deberás descargar el *dataset* utilizado desde el siguiente link y ubicar los archivos en la carpeta `código_eficiencia/data/`. Si ubicas los archivos bien, te debería quedar la siguiente estructura de archivos:

```
semana_08_programacion_funcional
│   1-paradigma_funcional.ipynb
│               ⋮
│
└──código_eficiencia
│   │   test_1_cargar_archivo.py
│   │               ⋮
│   │
│   └──data
│       │   credit_card_transactions.csv
│       │   states.csv
```

1. **Leer archivos línea por línea vs Leer el archivo completo.**

   Al momento de leer el contenido de un archivo existe tanto la posibilidad de revisar el contenido del archivo de apoco (`for line in file`) o cargar todo el contenido del archivo (`file.readlines()`). Veamos cómo se comporta el código cuando accedemos al contenido de distintas maneras.

```python
def cargar_archivo_generador() -> Generator:
    with open(ruta_transacciones, encoding='utf-8') as file:
        for line in file:
            yield line.strip().split(',')

def cargar_archivo_memoria() -> Generator:
    with open(ruta_transacciones, encoding='utf-8') as file:
        for line in file.readlines():
            yield line.strip().split(',')
```

Al ejecutar los códigos 10 veces y acceder a los primeros 100.000 datos, podemos observar los siguientes tiempos totales de ejecución: `txt Tiempo ejecución 'cargar_archivo_generador' 3.98 segundos Tiempo ejecución 'cargar_archivo_memoria' 10.80 segundos`

1. **Cargar los datos como `namedtuple` vs Cargar los datos como clases.**

   Otro punto importante a tener en consideración el momento de trabajar con grandes cantidades de datos, es ver si se cargará la información como una `namedtuple` o si se utilizarán clases personalizadas.

```
def cargar_namedtuples() -> Generator:
    with open(ruta_estados, encoding='utf-8') as file:
        for line in file:
            info = line.strip().split(';')
            yield NamedTupleEstado(*info)

def cargar_clases() -> Generator:
    with open(ruta_estados, encoding='utf-8') as file:
        for line in file:
            info = line.strip().split(';')
            yield ClaseEstado(*info)
```

Al ejecutar los códigos 10 veces, y acceder a los primeros 100000 datos, podemos observar los siguientes tiempos totales de ejecución son muy similares.

Pero recuerda que la diferencia entre usar `namedtuples` y clases no solo tiene efectos en los tiempo de ejecución, sino que también afecta la memoria que ocupa nuestro programa: `txt Uso memoria 'cargar_namedtuples' 25.37 MB Uso memoria 'cargar_clases' 28.43 MB`

1. **Agrupar generadores con funciones especializadas vs Agrupar generadores de forma manual.**

   En muchas ocasiones la información que debemos utilizar estará dividida en distintos generadores, por lo que deberemos agrupar la información, esta acción (u operación) es conocido como "*data join*". Veamos cómo cambia el comportamiento del código cuando utilizamos funciones especializadas en agrupar información, en comparación a hacerlo de forma manual.

```
def join_con_generadores(transacciones: Generator, estados:
Generator) -> filter:
    join_generadores = product(transacciones, estados)
    join_filtrado = filter(lambda x: x[0].state == x[1][0],
join_generadores)
    return join_filtrado
```

```
    def join_manual(transacciones: Generator, estados: Generator) ->
    list:
        dict_transacciones = defaultdict(list)
        list(map(lambda t: dict_transacciones[t.state].append(t),
    transacciones))
        join_anidado = (((t, e) for t in dict_transacciones[e[0]])
    for e in estados)
        join = reduce(lambda acc, t: acc + list(t), join_anidado, [])
        return join
```

Al ejecutar los códigos 10 veces podemos observar los siguientes tiempos totales de ejecución: `txt Tiempo ejecución 'join_con_generadores' 39.18 segundos Tiempo ejecución 'join_manual' 52.49 segundos`

En resumen, al momento de trabajar con estructuras Generadores es importante pensar en el código en términos de:

- Eficiencia
  - Cómo se carga la información
  - Cómo se maneja la información
- Comportamiento
  - Qué hacen las distintas funciones
  - Qué retornan las distintas funciones